LAPORAN PRAKTIKUM
JOBSHEET 09 - ABSTRACT CLASS DAN INTERFACE



Disusun Oleh:

Nama	:	Mohammad Hafidz Rafi' Rabbani
NIM	:	254107020084
Kelas	:	2H
No. Absen	:	14
Repository	:	github.com/hafidzrafi/java-oop

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

Daftar Isi

1. Tujuan 3

2. Praktikum 3

2.1. Percobaan 1: Abstract Class 3

2.1.1. Pertanyaan 5

2.2. Percobaan 2: Interface 7

2.2.1. Pertanyaan 9

2.3. Percobaan 3: Multiple Interfaces 11

2.3.1. Pertanyaan 13

3. Tugas 15

3.1. Tugas Mandiri: Sistem Klasifikasi Binatang (Abstract Class dan Multiple Interfaces) 15

1. Tujuan

- Menjelaskan tujuan arsitektural penggunaan abstract class dan interface dalam pemrograman berorientasi objek.
- Mengimplementasikan abstract class beserta abstract method pada hierarki pewarisan kelas (is-a relationship).
- Mengimplementasikan interface tunggal maupun multiple interfaces pada suatu kelas konkret (can-do relationship).
- Membedakan kapan harus menggunakan abstract class dan kapan harus menggunakan interface berdasarkan kebutuhan desain sistem.
- Menyimpan dan mendokumentasikan seluruh source code pekerjaan pada repositori GitHub resmi: https://github.com/hafidzrafi/java-oop.

2. Praktikum

2.1. Percobaan 1: Abstract Class

Pada percobaan pertama, diimplementasikan konsep **abstract class** melalui kelas induk `Hewan` yang mendefinisikan struktur dasar hewan peliharaan (`umur`, `bertambahUmur()`) serta kontrak perilaku abstrak `public abstract void bergerak();`. Kelas konkret `Kucing` dan `Ikan` mewarisi (`extends`) `Hewan` dan wajib memberikan implementasi spesifik pada method `bergerak()`. Kelas `Orang` bertindak sebagai klien yang berinteraksi secara polimorfik melalui referensi bertipe `Hewan`.

```
// Hewan.java
package jobsheet.jobsheet09.src.experiments.exp1;

public abstract class Hewan {
    protected int umur;

    protected Hewan() {
        this.umur = 0;
    }

    public void bertambahUmur() {
        this.umur += 1;
    }

    public abstract void bergerak();
}
```

```java
// Kucing.java & Ikan.java
package jobsheet.jobsheet09.src.experiments.exp1;

public class Kucing extends Hewan {
    @Override
    public void bergerak() {
        System.out.println("Walking on LEGS, \"tap..tap..\"");
    }
}

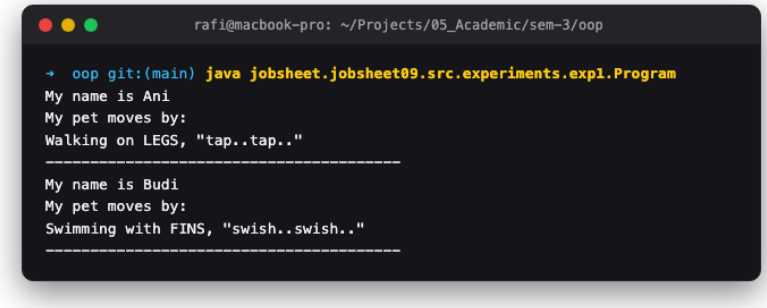
public class Ikan extends Hewan {
    @Override
    public void bergerak() {
        System.out.println("Swimming with FINS, \"swish..swish..\"");
    }
}
```

```java
// Orang.java & Program.java
package jobsheet.jobsheet09.src.experiments.exp1;

public class Orang {
    private String nama;
    private Hewan hewanPeliharaan;

    public Orang(String nama) { this.nama = nama; }
    public void peliharaHewan(Hewan hewan) { this.hewanPeliharaan = hewan; }

    public void ajakPeliharaanJalanJalan() {
        System.out.println("My name is " + this.nama);
        System.out.println("My pet moves by: ");
        this.hewanPeliharaan.bergerak();
        System.out.println("----------------------------------------");
    }
}
```



2.1.1. Pertanyaan

1. Explain how the program above runs. Why does the call `this.hewanPeliharaan.bergerak()` in the class `Orang` produce different output for Ani and Budi?

   - Kedua objek `ani` dan `budi` merupakan instance dari kelas `Orang` yang memiliki atribut referensi polimorfik `private Hewan hewanPeliharaan`. Pada objek `ani`, atribut `hewanPeliharaan` menyimpan alamat memori objek konkret `Kucing` di Heap, sedangkan pada `budi` menyimpan alamat memori objek konkret `Ikan`. Saat `this.hewanPeliharaan.bergerak()` dieksekusi, instruksi bytecode `invokevirtual` melakukan **dynamic method dispatch** dengan membaca **Klass Pointer** pada Object Header di Heap, lalu memanggil implementasi `bergerak()` pada **virtual method table (vtable)** masing-masing subclass (`Kucing.bergerak()` untuk Ani dan `Ikan.bergerak()` untuk Budi).

2. Add the line `Hewan hewan = new Hewan();` to the `main()` method. Show the compilation result and explain it briefly.

   - Hasil kompilasi (`javac`): `error: Hewan is abstract; cannot be instantiated`. Kelas `Hewan` dideklarasikan dengan modifier `abstract` (flag bytecode `ACC_ABSTRACT`). Sesuai spesifikasi JVM (JVMS §4.1, §6.5 `new`), kelas abstrak tidak dapat diinstansiasi secara langsung menjadi objek konkret di Heap karena memiliki metode abstrak (`bergerak()`) yang belum memiliki blok instruksi kode (**method body**) pada tabel metode virtualnya.

3. Is a class that extends an abstract class allowed not to implement the abstract methods of its parent? Prove it by removing the method `bergerak()` from the class `Ikan`, then show the compilation result and explain it.

   - Hasil kompilasi (`javac`): `error: Ikan is not abstract and does not override abstract meth in Hewan`

     Suatu concrete class yang mewarisi (`extends`) abstract class wajib mengimplementasikan seluruh method abstrak milik superclass-nya. Pengecualian (JLS §8.1.1.1) hanya berlaku apabila subclass tersebut juga dideklarasikan sebagai `abstract class`. Karena `Ikan` adalah concrete class, penghapusan method `bergerak()` melanggar kontrak pewarisan dan gagal dikompilasi.

4. Add the abstract method `public abstract void bernapas();` inside the class `Ikan` without changing the declaration of the class `Ikan`. Show the compilation result and explain which rule is violated.

   - Hasil kompilasi (`javac`): `error: Ikan is not abstract and does not override abstract meth in Ikan`

     Aturan yang dilanggar adalah JLS §8.4.3.1: method abstrak (**abstract method**) hanya boleh dideklarasikan di dalam `abstract class` atau `interface`. Suatu kelas konkret (`public class Ikan`) tidak diizinkan memiliki method tanpa body implementasi.

2.2. Percobaan 2: Interface

Pada percobaan kedua, diimplementasikan kontrak `interface ICumlaude` yang mendefinisikan syarat kelulusan cumlaude (`lulus()` dan `meraihIPKTinggi()`). Kelas `Sarjana` dan `PascaSarjana` mewarisi kelas `Mahasiswa` (`extends Mahasiswa`) sekaligus mengimplementasikan kontrak `ICumlaude` (`implements ICumlaude`). Kelas `Rektor` menerima parameter bertipe interface `ICumlaude` pada method `beriSertifikatCumlaude()`.

```
// ICumlaude.java
package jobsheet.jobsheet09.src.experiments.exp2;

public interface ICumlaude {
    public abstract void lulus();
    public abstract void meraihIPKTinggi();
}
```

```
// Sarjana.java & PascaSarjana.java
package jobsheet.jobsheet09.src.experiments.exp2;

public class Sarjana extends Mahasiswa implements ICumlaude {
    public Sarjana(String nama) { super(nama); }

    @Override
    public void lulus() {
        System.out.println("I have completed my UNDERGRADUATE THESIS");
    }

    @Override
    public void meraihIPKTinggi() {
        System.out.println("My GPA is above 3.51");
    }
}

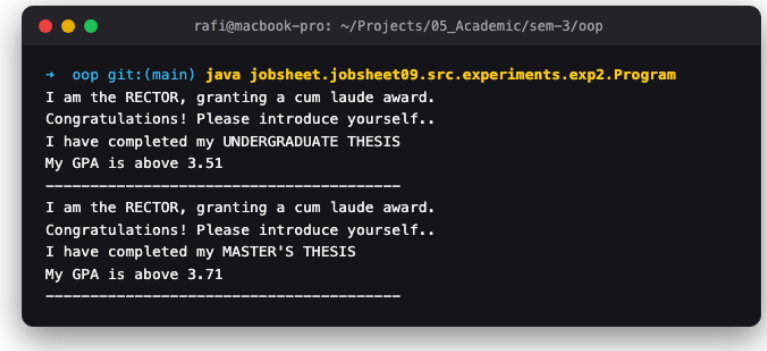
public class PascaSarjana extends Mahasiswa implements ICumlaude {
    public PascaSarjana(String nama) { super(nama); }

    @Override
    public void lulus() {
        System.out.println("I have completed my MASTER'S THESIS");
    }

    @Override
    public void meraihIPKTinggi() {
        System.out.println("My GPA is above 3.71");
    }
}
```

```java
// Rektor.java & Program.java
package jobsheet.jobsheet09.src.experiments.exp2;

public class Rektor {
    public void beriSertifikatCumlaude(ICumlaude mahasiswaCumlaude) {
        System.out.println("I am the RECTOR, granting a cum laude award.");
        System.out.println("Congratulations! Please introduce yourself..");
        mahasiswaCumlaude.lulus();
        mahasiswaCumlaude.meraihIPKTinggi();
        System.out.println("----------------------------------------");
    }
}
```



2.2.1. Pertanyaan

1. Why does an error occur in step 7 (`pakRektor.beriSertifikatCumlaude(mhsBiasa);`)? Write down the error message and explain how you fixed it.

   - Pesan error (`javac`): `error: incompatible types: Mahasiswa cannot be converted to ICumlaude`.

     Parameter formal pada method `beriSertifikatCumlaude(ICumlaude mahasiswaCumlaude)` membutuhkan referensi objek yang mengimplementasikan tipe kontrak `ICumlaude`. Objek `mhsBiasa` bertipe `Mahasiswa` (superclass dasar) yang tidak mengimplementasikan interface `ICumlaude`. Perbaikan dilakukan dengan mengomentari/menghapus baris pemanggilan `pakRektor.beriSertifikatCumlaude(mhsBiasa);`.

2. Can the method `kuliahDiKampus()` be called from the object `sarjanaCumlaude` in the class `Program`? Why?

   - Bisa. Variabel `sarjanaCumlaude` dideklarasikan dengan static type `Sarjana`. Karena `Sarjana` merupakan subclass dari `Mahasiswa` (`public class Sarjana extends Mahasiswa`), maka `Sarjana` mewarisi method `public void kuliahDiKampus()` dari superclass `Mahasiswa` sehingga simbol method tersebut terdaftar di tabel simbol kompilator maupun vtable runtime.

3. Can the method `kuliahDiKampus()` be called from the parameter `mahasiswaCumlaude` in the method `beriSertifikatCumlaude()` of the class `Rektor`? Why?

   - Tidak bisa secara langsung. Parameter `mahasiswaCumlaude` memiliki static type descriptor `ICumlaude` (tipe interface). Pada fase **compile-time symbol resolution**, compiler hanya mengizinkan pemanggilan method yang dideklarasikan di dalam `interface ICumlaude` (`lulus()` dan `meraihIPKTinggi()`). Karena `kuliahDiKampus()` milik kelas `Mahasiswa` dan bukan bagian dari kontrak `ICumlaude`, pemanggilan langsung akan menghasilkan compile error `cannot find symbol` (kecuali dilakukan eksplisit **downcasting** ke `(Mahasiswa)`).

4. Remove the keyword `public` from the method `lulus()` in the class `Sarjana`. Show the compilation result and explain it, then restore the code.

   - Hasil kompilasi (`javac`): `error: lulus() in Sarjana cannot implement lulus() in ICumlaud to assign weaker access privileges; was public`

     Berdasarkan JLS §9.4, semua method abstrak di dalam `interface` secara implisit memiliki modifier `public abstract`. Berdasarkan aturan overriding dan implementasi interface (JLS §8.4.8.3), kelas pengimplementasi dilarang mempersempit visibilitas (**weaker access privilege**) dari method kontraknya. Menghapus kata kunci `public` menurunkan visibilitas `lulus()` di `Sarjana` menjadi **package-private** (default), sehingga ditolak oleh compiler.

2.3. Percobaan 3: Multiple Interfaces

Pada percobaan ketiga, ditambahkan interface kedua yaitu `IBerprestasi` (`menjuaraiKompetisi()` dan `membuatPublikasiIlmiah()`). Kelas `PascaSarjana` dan `Sarjana` mengimplementasikan dua interface sekaligus (`implements ICumlaude, IBerprestasi`) untuk mendemonstrasikan kapabilitas **multiple interface implementation** di Java sebagai pengganti multiple class inheritance.

```
// IBerprestasi.java
package jobsheet.jobsheet09.src.experiments.exp3;

public interface IBerprestasi {
    public abstract void menjuaraiKompetisi();
    public abstract void membuatPublikasiIlmiah();
}
```

```
// Sarjana.java & PascaSarjana.java (Multiple Interfaces)
package jobsheet.jobsheet09.src.experiments.exp3;

public class Sarjana extends Mahasiswa implements ICumlaude, IBerprestasi {
    public Sarjana(String nama) { super(nama); }

    @Override
    public void lulus() {
        System.out.println("I have completed my UNDERGRADUATE THESIS");
    }

    @Override
    public void meraihIPKTinggi() {
        System.out.println("My GPA is above 3.51");
    }

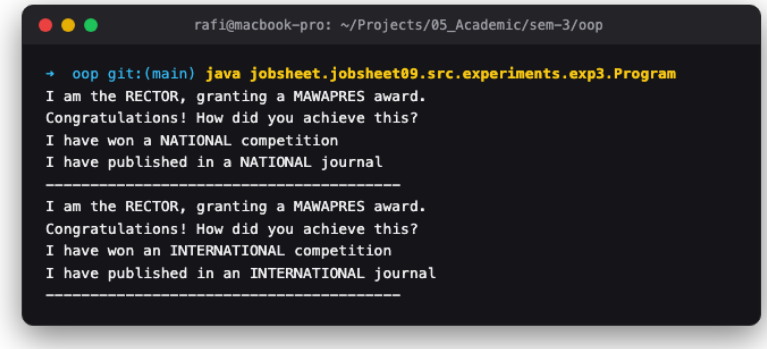
    @Override
    public void menjuaraiKompetisi() {
        System.out.println("I have won a NATIONAL competition");
    }

    @Override
    public void membuatPublikasiIlmiah() {
        System.out.println("I have published in a NATIONAL journal");
    }
}
```

```
// Program.java
package jobsheet.jobsheet09.src.experiments.exp3;
```

```java
public class Program {
    public static void main(String[] args) {
        Rektor pakRektor = new Rektor();
        Sarjana sarjanaCumlaude = new Sarjana("Dini");
        PascaSarjana masterCumlaude = new PascaSarjana("Elok");

        pakRektor.beriSertifikatMawapres(sarjanaCumlaude);
        pakRektor.beriSertifikatMawapres(masterCumlaude);
    }
}
```

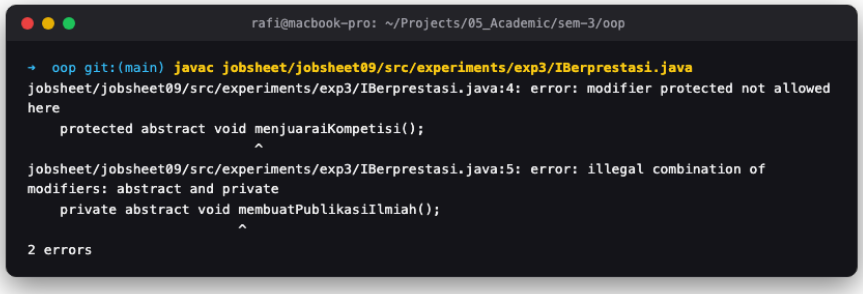


2.3.1. Pertanyaan

1. In the interface `IBerprestasi`, change the `public` modifier of the method `menjuaraiKompetisi()` to `protected`, and that of the method `membuatPublikasiIlmiah()` to `private`. What happens? Explain why (include a screenshot of the result), then restore the code.

   - Kompilasi gagal dengan 2 pesan error dari `javac`:

     1. `error: modifier protected not allowed here` pada `protected abstract void menjuaraiKompetisi();`. Modifier `protected` dilarang di dalam interface (JLS §9.4) karena `protected` mengikat visibilitas pada pewarisan hierarki kelas (`extends`), sedangkan interface adalah kontrak publik lintas hierarki (`implements`).
     2. `error: illegal combination of modifiers: abstract and private` pada `private abstract void membuatPublikasiIlmiah();`. Meskipun sejak Java 9 interface mengizinkan helper method `private` yang memiliki body implementasi `{ ... }`, method abstrak (`abstract`) tidak pernah boleh dideklarasikan `private`. Kata kunci `abstract` mewajibkan kelas pengimplementasi meng-override method tersebut pada **interface method table (`itable`)**, sedangkan `private` mengunci akses method hanya di dalam scope file interface itu sendiri (kontradiksi semantik).



2. Add the line `IBerprestasi prestasi = new IBerprestasi();` to the `main()` method. Why does this line cause an error?

   - Hasil kompilasi (`javac`): `error: IBerprestasi is abstract; cannot be instantiated`. `IBerprestasi` adalah sebuah `interface` (memiliki flag bytecode `ACC_INTERFACE` dan

`ACC_ABSTRACT`). Interface tidak memiliki konstruktor (`<init>`), tidak memiliki instance field di Heap, dan method abstraknya tidak memiliki implementasi kode bytecode. Oleh sebab itu, instruksi `new` tidak dapat mengalokasikan objek konkret dari tipe interface secara langsung.

3. If an outstanding Sarjana must win a NATIONAL competition and publish in a NATIONAL journal, modify the related classes so that in the class `Program` the object `pakRektor` can present the Mawapres award to the object `sarjanaCumlaude`, with the expected output.

   - Modifikasi dilakukan dengan menambahkan `IBerprestasi` pada deklarasi kelas `Sarjana` (`public class Sarjana extends Mahasiswa implements ICumlaude, IBerpres` mengimplementasikan method `menjuaraiKompetisi()` dan `membuatPublikasiIlmiah()` dengan output berlingkup `"NATIONAL"`, lalu memanggil `pakRektor.beriSertifikatMawapres(sarjanaCumlaude);` di dalam `Program.main()`. Hasil eksekusinya telah ditampilkan pada screenshot output Percobaan 3 di atas.

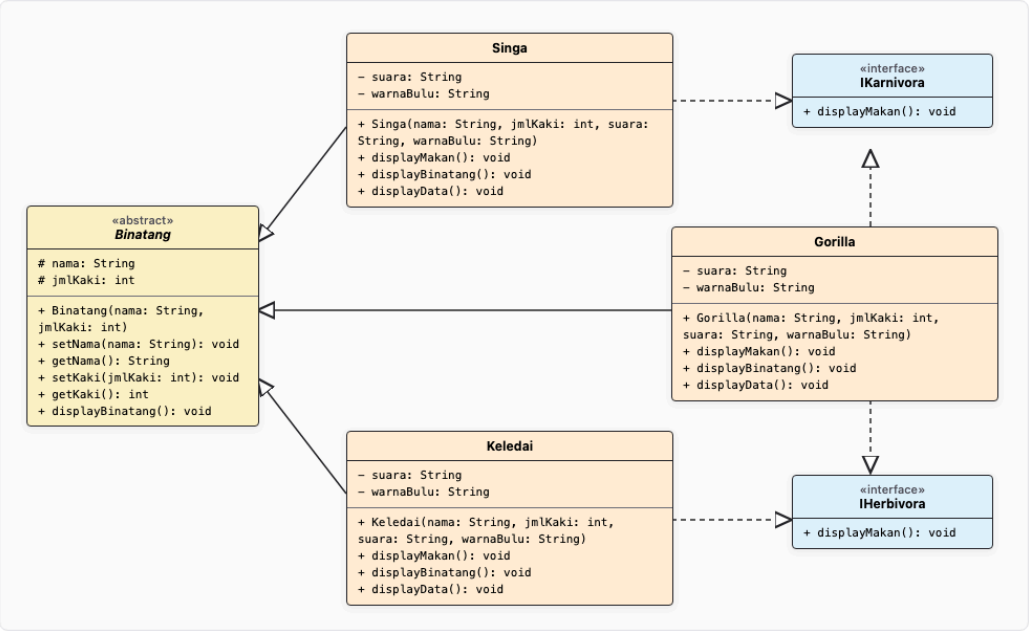
3. Tugas

3.1. Tugas Mandiri: Sistem Klasifikasi Binatang (Abstract Class dan Multiple Interfaces)

Tugas mandiri mengimplementasikan sistem identifikasi karakteristik fisik dan jenis makanan hewan yang menggabungkan **abstract class** (relasi **is-a**) dan **multiple interfaces** (relasi **can-do**):

- `Binatang`: Abstract class yang menyimpan atribut umum `#nama: String` dan `#jmlKaki: int`, konstruktor berparameter, getter/setter, serta method `displayBinatang()`.
- `IKarnivora` & `IHerbivora`: Interface yang masing-masing mendeklarasikan method kontrak `public abstract void displayMakan();`.
- `Keledai`: Subclass `Binatang` yang mengimplementasikan `IHerbivora`.
- `Singa`: Subclass `Binatang` yang mengimplementasikan `IKarnivora`.
- `Gorilla`: Subclass `Binatang` yang mengimplementasikan dua interface sekaligus (`implements IKarnivora, IHerbivora`). Karena kedua interface memiliki method dengan signature yang identik (`void displayMakan()`), `Gorilla` cukup menyediakan satu implementasi `displayMakan()` yang memenuhi kedua kontrak **interface method table (`itable`)** sekaligus.

Diagram Kelas UML Sistem Binatang:



```java
// Binatang.java
package jobsheet.jobsheet09.src.assignments;

public abstract class Binatang {
    protected String nama;
    protected int jmlKaki;

    public Binatang(String nama, int jmlKaki) {
        this.nama = nama;
        this.jmlKaki = jmlKaki;
    }

    public void setNama(String nama) { this.nama = nama; }
    public String getNama() { return this.nama; }
    public void setKaki(int jmlKaki) { this.jmlKaki = jmlKaki; }
    public int getKaki() { return this.jmlKaki; }

    public void displayBinatang() {
        System.out.println("Name: " + this.nama);
        System.out.println("Number of Legs: " + this.jmlKaki);
    }
}
```

```java
// IKarnivora.java & IHerbivora.java
package jobsheet.jobsheet09.src.assignments;

public interface IKarnivora {
    public abstract void displayMakan();
}

public interface IHerbivora {
    public abstract void displayMakan();
}
```

```java
// Keledai.java, Gorilla.java, & Singa.java
package jobsheet.jobsheet09.src.assignments;

public class Gorilla extends Binatang implements IKarnivora, IHerbivora {
    private String suara;
    private String warnaBulu;

    public Gorilla(String nama, int jmlKaki, String suara, String warnaBulu) {
        super(nama, jmlKaki);
        this.suara = suara;
        this.warnaBulu = warnaBulu;
    }

    @Override
    public void displayMakan() {
```

```
        System.out.println("Type: Carnivore + Herbivore");
        System.out.println("Food: Meat + Plants");
    }

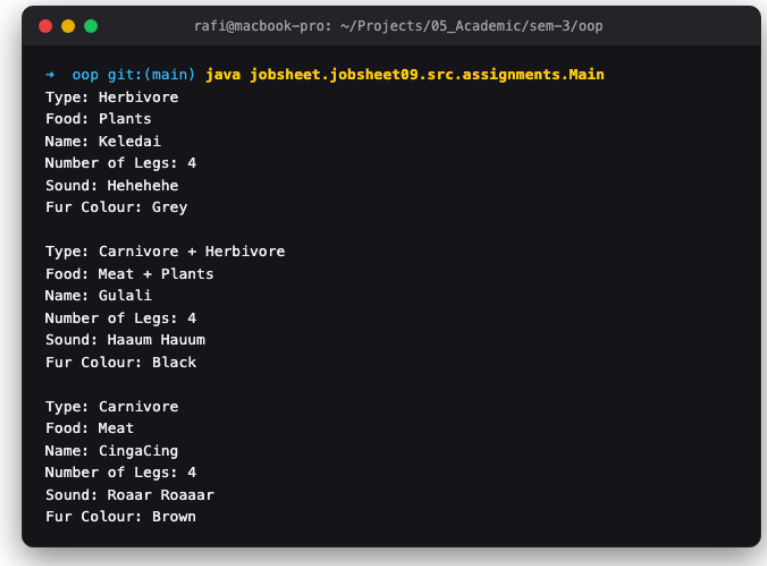
    @Override
    public void displayBinatang() {
        System.out.println("Name: " + this.nama);
        System.out.println("Number of Legs: " + this.jmlKaki);
        System.out.println("Sound: " + this.suara);
        System.out.println("Fur Colour: " + this.warnaBulu);
    }

    public void displayData() {
        this.displayMakan();
        this.displayBinatang();
        System.out.println();
    }
}
```

```
// Main.java
package jobsheet.jobsheet09.src.assignments;

public class Main {
    public static void main(String[] args) {
        Keledai keledai = new Keledai("Keledai", 4, "Hehehehe", "Grey");
        Gorilla gorilla = new Gorilla("Gulali", 4, "Haaum Hauum", "Black");
        Singa singa = new Singa("CingaCing", 4, "Roaar Roaaar", "Brown");

        keledai.displayData();
        gorilla.displayData();
        singa.displayData();
    }
}
```


rafi@macbook-pro: ~/Projects/05_Academic/sem-3/oop
→ oop git:(main) java jobsheet.jobsheet09.src.assignments.Main
Type: Herbivore
Food: Plants
Name: Keledai
Number of Legs: 4
Sound: Hehehehe
Fur Colour: Grey
Type: Carnivore + Herbivore
Food: Meat + Plants
Name: Gulali
Number of Legs: 4
Sound: Haaum Hauum
Fur Colour: Black
Type: Carnivore
Food: Meat
Name: CingaCing
Number of Legs: 4
Sound: Roaar Roaaar
Fur Colour: Brown